

Creating IaC from infrastructure services and cloud resources can be done using a reverse engineering tool like TerraCognita, which was developed as an open source facility by Cycloid. TerraCognita can be used to develop IaC code from live cloud accounts in AWS, Azure or Google and from private cloud setups in OpenStack and VMWare. Cycloid has multiple open source projects including TerraCognita, InfraMap (for cloud diagrams) and TerraCost (for predictive cloud cost estimation). TerraCognita gets IaC scripts as Terraform templates from infrastructure services and uses InfraImport to get this infrastructure service converted as code.

Why we need Infrastructure as Code

During cloud adoption, we have repeated activity for infrastructure provisioning, which could be simple or complex when it comes to multiple environment provisioning. Hence, automating and reducing the manual effort in such activities (including compute, storage, network and security configurations) helps to reduce overall DevOps operational activities and improve efficiency in engineering activities (Figure 1).

This kind of IaC can be available as a native facility in Azure as Azure Resource Manager (ARM) templates, in AWS as CloudFormation templates and in GCP as Google Deployment Manager (GDM). All these are in JSON format to prepare scripts for infrastructure provisioning and configuration.

For instance, an AWS feature like CloudFormation accomplishes this by defining provisioning code in a script using Typescript or Python for resource development and deployment. With a cloud development kit (CDK), this CloudFormation is taken to the next level of IaC. By using CDK, we can

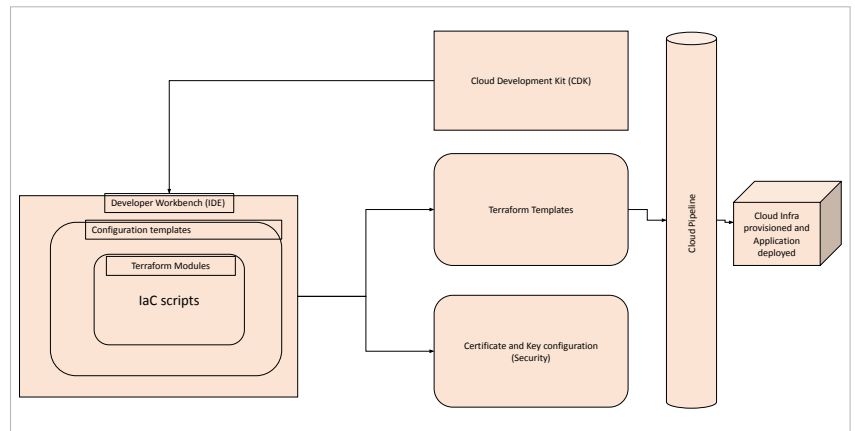


Figure 1: Components for developing IaC using Terraform templates

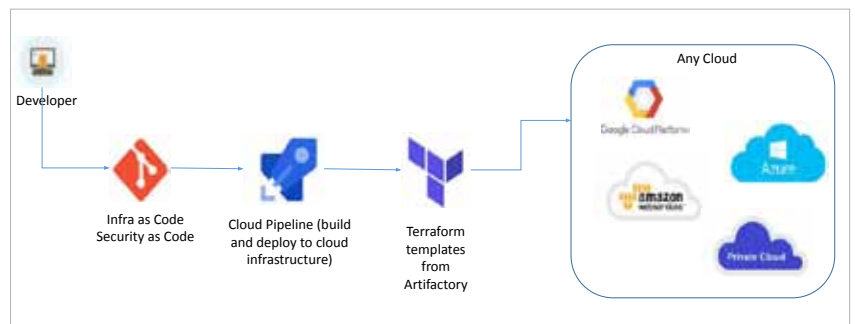


Figure 2: Implementation flow using Terraform templates

develop AWS constructs embedding CloudFormation templates, and configure services and business logic in native code using Java, Python, JavaScript or Typescript.

When we have a large number of microservices in terms of containers and serverless applications, it would be a Herculean task to handle code build, deployment, infrastructure provisioning, and platform management of cloud services across different stages, monitoring and management activities, and platform updates.

Platform teams automate many of these activities through Infrastructure as Code (IaC), but it is still a complex activity to manage the cloud and its operations in large enterprises. In the case of Microsoft Azure, using a templated solution with Azure Resource Manager (ARM) helps to replicate configuration and application infrastructure provisioning. Here,

pre-defined code scripts are used for building a descriptive model that defines VMs, networks, security and connectors easily.

Azure DevOps can be integrated with the ARM group for secure deployment of applications to different stages by selective users, and for tag deployment and code promotion with different parallel releases. This is so that we can promote (when regression checks are successful) or demote (when there is a reason to rollback to the previous release) the code build, as needed, quickly. It can also be used to store reusable artifacts like library or container images, so that it can be reused for deployment hooks without being built again.

Foundational services of Terraform

When we think of an agnostic solution for IaC in a multi-cloud environment, we can think of HashiCorp's Terraform